

Docker Volume

أي App عبارة عن Data + Process

- DNS عبارة عن Service بتعمل بعض الـ Process من خلال بعض الـ Data عندها
- لو مفيش Data الـ Process مش هتشتغل
- # لما يكون عندنا Container وفيه App وحطيت الـ Data في الـ Container
- لو حطيت الـ Data في أكثر من Container كل Container عبارة عن virtual isolated بالتالي لما احط الـ Data في كل container هفقد عنصر المزامنه (Synchronization)
- يحصل تكرار للـ Data بالتالي يحصل Duplication
- هواجه صعوبة عاليه جدا في الـ Backup لأنني هأخذ الـ Backup من انهي Container
- هواجه صعوبة عاليه جدا في الـ Security

قاعدة عامة = الـ Security و الـ Performance علاقة عكسية

- كلما زاد الـ Security كلما قل الـ Performance
- بالتالي لما أكون عايز أنقل الـ Data واشفر الـ Data هياخد وقت كثير جدا ولما أفك التشفير برضو هأخذ وقت
- لو عايز أنقل الـ container من مكانه هياخد وقت كبير جدا لأن مساحة الـ Data هتكون عاليه جدا
- اهم حاجه للـ App الـ Availability يعني يكون الـ Data دايمًا موجوده وقت لما احتاجها دايمًا كل حاجه جاهزة
- وارد جدا الـ Container يتحذف
- هفقد عنصر الـ Accessibility يعني قدرة الوصول للـ Data في أي وقت ومن أي مكان لو الـ Container اتعمله Stop ساعتها هفقد عتصر الـ Accessibility

يحصل مشاكل في اهم العناصر

- 1. Accessibility
- 2. Synchronization
- 3. Availability
- 4. Security
- 5. Backup
- 6. Migration
- 7. Duplication

في حالة لو اعتمدت وجود الـ Data داخل الـ Container

الحل هو فصل الـ Data عن الـ Container فالحالة دي المشاكل هتكون مزاي

- هنواجه مشكله اننا هنفصل الـ Data عن الـ Container وهو أصلا Virtual Isolated ساعتها الـ Docker Volume اللي هيجل المشكله دي

الـ Docker Volume هو المسؤول عن فصل الـ Data عن الـ Container وعمل Link بين الـ Container و الـ Data

الـ Docker Volume جايه منين

- هكتب أمر

```
yousef@DevOpsVM:~$ sudo docker info
```

- هلاقي في خانة اسمها plugins

```
Plugins:
Volume: local
Network: bridge host ipvlan macvlan null overlay
Log: awslogs fluentd gcplogs gelf journald json-file local splunk syslog
CDI spec directories:
/etc/cdi
/var/run/cdi
Swarm: inactive
```

- الـ Plugins عبارة عن Exinctions إضافات في الـ Docker او ممكن تعتبره features انت بتضيفها وفي Built in

هنعمل سيناريو لفهم كل التفاصيل

Lab1 -

1. Create Volume

2. Check / List Volume

3. Create Container + Map Volume To Container

4. check Volume / Edit

5. Test

Lab2 -

6. Stop Container

7. Check Volume / Edit

8. Start Container

9. Test

Lab3 -

10. Stop / Delete Container

11. Check Volume / Edit

12. Create new Container → Map Container → Volume

13. Test

Lab4 -

14. Create new Container → Map Container → Volume

15. Test

16. Edit volume

17. Test two Containers

الـ **Volume** يعني المكان اللي بـخزن فيه الـ **Data**

يبقى أول خطوة اعمل المكان اللي هحط فيه الـ **Data**

1. Create Volume

- هـستعلم عن الـ **volumes** الموجوده

```
yousef@DevOpsVM:~$ sudo docker volume ls
DRIVER          VOLUME NAME
```

- هـنشئ **volume** واستعلم عن الـ **Volumes**

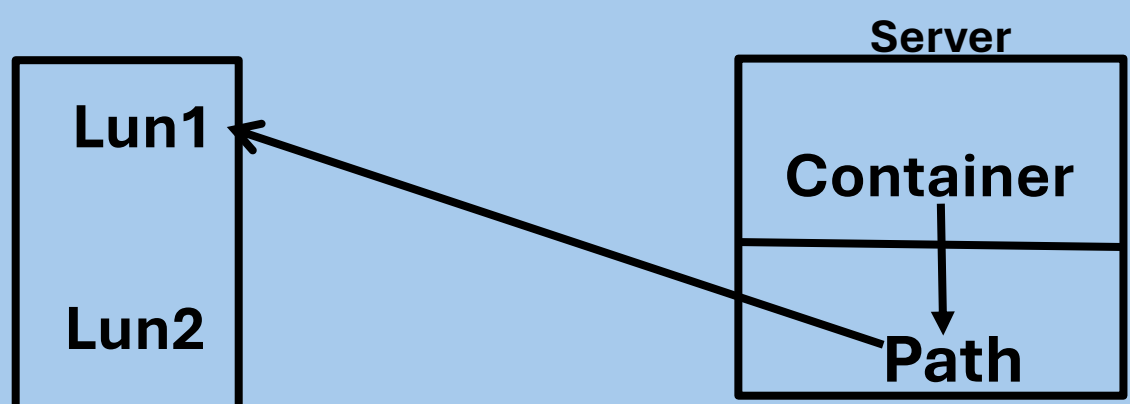
```
yousef@DevOpsVM:~$ sudo docker volume create appdata
appdata
yousef@DevOpsVM:~$ sudo docker volume ls
DRIVER          VOLUME NAME
local           appdata
```

- هـستعلم عن الـ **Volume** اللي عملناه

```
yousef@DevOpsVM:~$ sudo docker volume inspect appdata
[
  {
    "CreatedAt": "2026-02-10T15:35:59+02:00",
    "Driver": "local",
    "Labels": null,
    "Mountpoint": "/var/lib/docker/volumes/appdata/_data",
    "Name": "appdata",
    "Options": null,
    "Scope": "local"
  }
]
```

- في عندنا خانة باسم (**Mountpoint**) فيها المكان اللي هـيتخزن فيها الـ **Data**

- الـ **Data** بـتتخزن بالشكل دا



- الـ **Container** بيخزن الـ **Data** في مسار معين (**path**) واربطه بـ **Lun** يعني بحجز مكان في الـ **Hard** كأنه بارتيشن

بالتالي كذا الـ **Data** أصبحت منفصله عن الـ **container** واتعملها **Link**

- لو الـ **Server** وقع أقدر بكل بساطة اخذ نفس الـ **Image** واعمـل بيها نفس الـ **Container** واربطها مع الـ **Data** ثاني

2. Check / List Volume

- هنشوف المكان اللي فيه الـ **Data** هو الـ **Mountpoint**

```
yousef@DevOpsVM:~$ sudo ls --color /var/lib/docker/volumes/appdata/_data
yousef@DevOpsVM:~$ sudo ls --color /var/lib/docker/volumes/appdata/_data
yousef@DevOpsVM:~$ sudo ls --color /var/lib/docker/volumes/appdata/_data
yousef@DevOpsVM:~$ sudo ls --color /var/lib/docker/volumes/appdata/_data
yousef@DevOpsVM:~$ sudo ls --color /var/lib/docker/volumes/appdata/_data
yousef@DevOpsVM:~$ |
```

- مفيش أي **Data** ظاهره لأن مفيش لسا أصلا **Data** اتضافت

3. Create Container + Map Volume To Container

```
yousef@DevOpsVM:~$ sudo docker run -d --name cont_vol2 -p 93:80
--volume appdata:/usr/local/apache2/htdocs myapp:v1
```

- الـ **usr/local/apache2/htdocs/** : دا مسار الـ **Data** في الـ **Image**

- ازاي اعرف مكان الـ **Data**

- لازم اعرف الـ **Image** معموله على انه **Software**

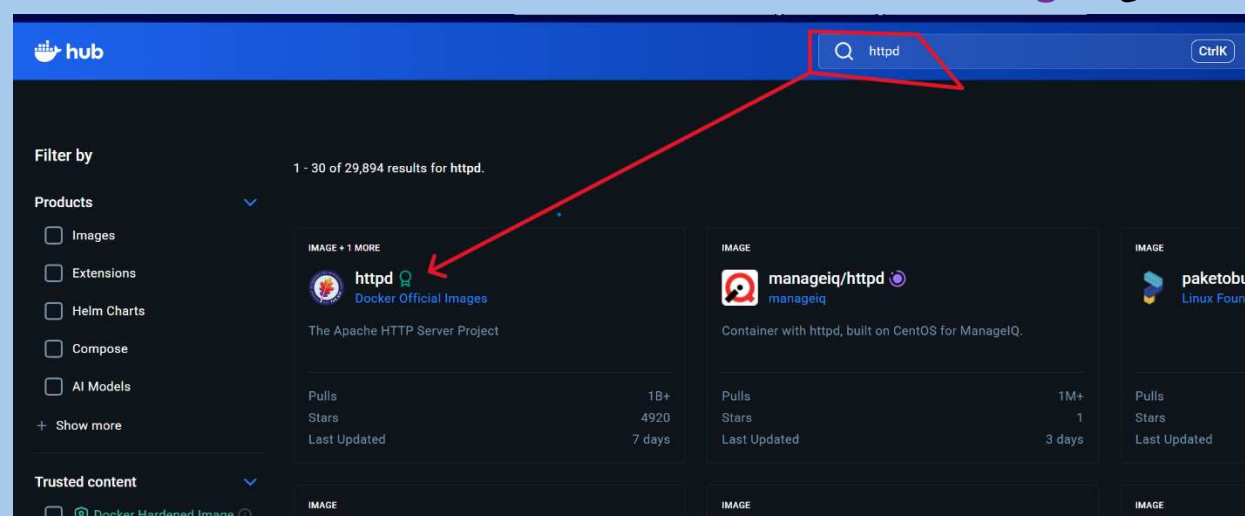
```
yousef@DevOpsVM:~$ sudo docker inspect myapp:v1
[
  {
    "Architecture": "amd64",
    "Config": {
      "Cmd": [
        "httpd-foreground"
      ],
      "Env": [
        "PATH=/usr/local/apache2/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin",
        "HTTPD_PREFIX=/usr/local/apache2",
        "HTTPD_VERSION=2.4.66",
        "HTTPD_SHA256=94d7ff2b42acbb828e870ba29e4cbad48e558a79c623ad3596e4116efcfea25a",
        "HTTPD_PATCHES="
      ],
      "ExposedPorts": {
        "80/tcp": {}
      },
      "StopSignal": "SIGWINCH",
      "WorkingDir": "/usr/local/apache2"
    },
    "Created": "2026-02-08T08:07:13.970650339+02:00",
    "Descriptor": {
      "digest": "sha256:fdd3336086b101af6bf1a6bbb62fade9f3bcdeec6666b6791dff50db8e710b62"
    }
  }
]
```

- كدا الـ **apache2 = myapp:v1**

- هدخل على **Docker hup**

- ابحت عن **httpd**

- ادخل على الـ **image** الرسميه



▪ دلوقتي هنلاقي ال document فيها ال path اللي فيه ال Data

Create a Dockerfile in your project

```
FROM httpd:2.4
COPY ./public-html/ /usr/local/apache2/htdocs/
```

Then, run the commands to build and run the Docker image:

```
$ docker build -t my-apache2 .
$ docker run -dit --name my-running-app -p 8080:80 my-apache2
```

Visit <http://localhost:8080> and you will see It works!

Without a Dockerfile

If you don't want to include a Dockerfile in your project, it is sufficient to do the following:

```
$ docker run -dit --name my-apache-app -p 8080:80 -v "$PWD":/usr/local/apache2/htdocs/ httpd:2.4
```

Configuration

To customize the configuration of the httpd server, first obtain the upstream default configuration from the container:

```
$ docker run --rm httpd:2.4 cat /usr/local/apache2/conf/httpd.conf > my-httpd.conf
```

You can then COPY your custom configuration in as /usr/local/apache2/conf/httpd.conf :

```
FROM httpd:2.4
COPY ./my-httpd.conf /usr/local/apache2/conf/httpd.conf
```

check volume / edit .4

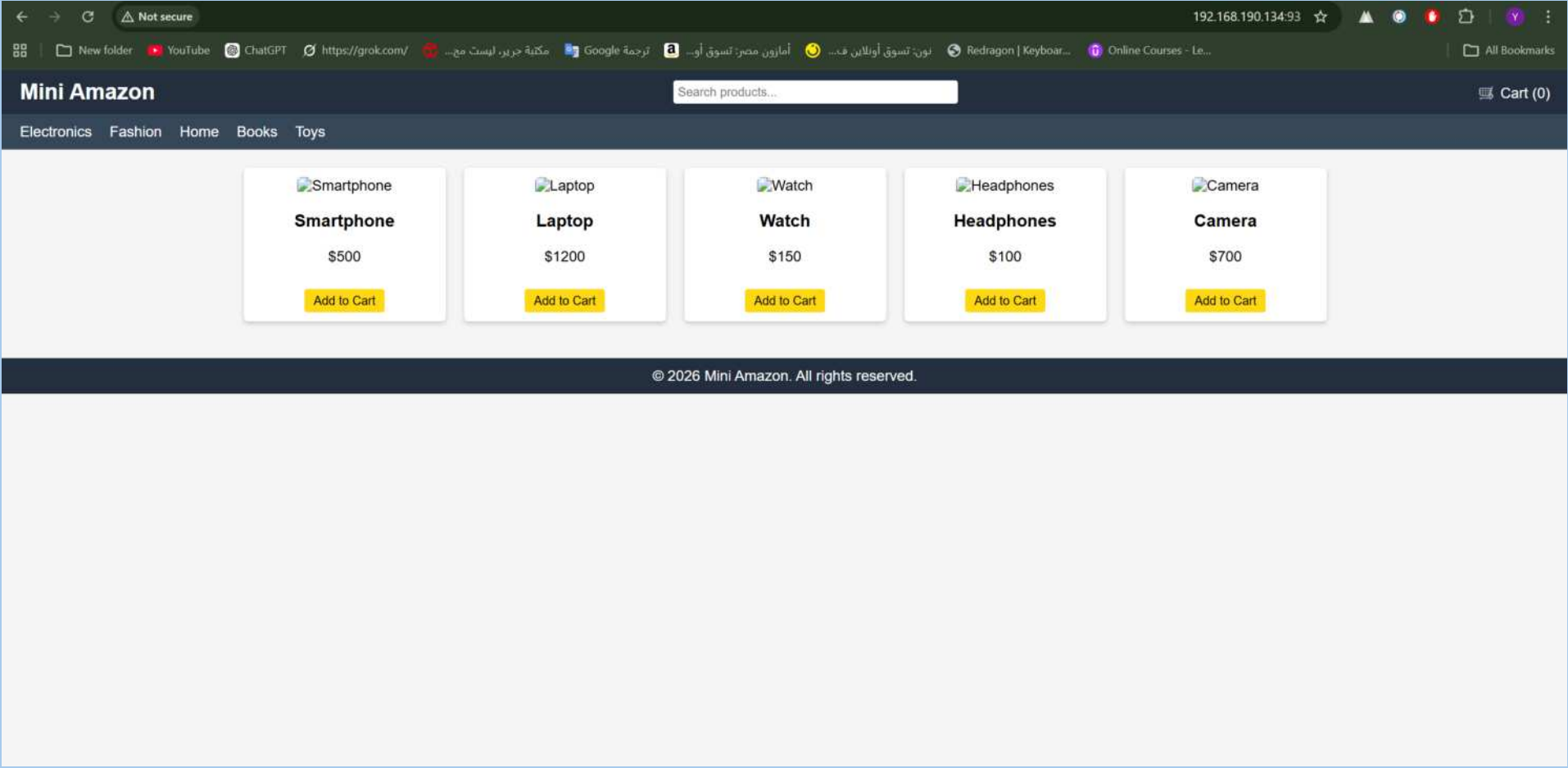
yousef@DevOpsVM:~\$ sudo vim /var/lib/docker/volumes/appdata/_data/index.html|

- كذا احنا هندخل على ال Data ونعدل فيها وهتسمع في ال container

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Mini Amazon</title>
  <style>
    body { font-family: Arial, sans-serif; margin:0; padding:0; background:#f5f5f5; }
    header { background:#232f3e; color:white; padding:10px 20px; display:flex; align-items:center; justify-content:space-between; }
    header .logo { font-size:24px; font-weight:bold; }
    header input[type="text"] { padding:5px; width:300px; border-radius:3px; border:none; }
    header .cart { cursor:pointer; }
    nav { background:#37475a; padding:10px 20px; color:white; }
    nav a { color:white; margin-right:15px; text-decoration:none; }
    .container { display:flex; flex-wrap:wrap; padding:20px; justify-content:center; }
    .product { background:white; width:200px; border-radius:5px; padding:10px; box-shadow:0 0 0 0.2px rgba(0,0,0,0.2); text-align:center; }
    .product img { width:100%; height:150px; object-fit:cover; border-radius:5px; }
    .product button { background:#ffd814; border:none; padding:5px 10px; margin-top:10px; }
```

- قمت بأضافة ال Data

- عايزين نشوفها



Stop Containers .5

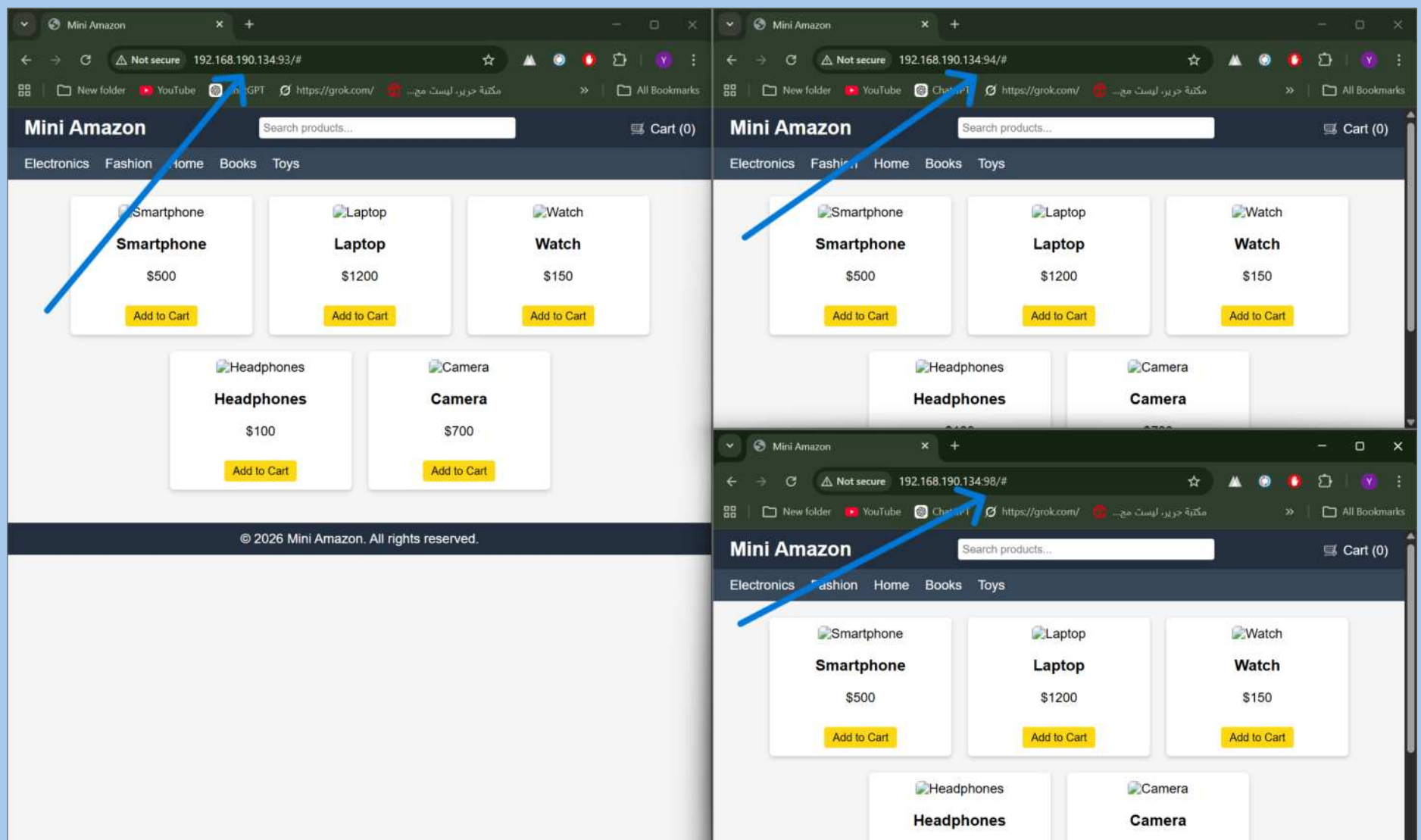
```
yousef@DevOpsVM:~$ sudo docker stop cont_vol2  
cont_vol2
```

- اقدر بكل بساطه اعدل على ال Data براحتي والـ Container متوقف

Synchronization .6

- يعني في اكثر من حاجه شغاله مع بعض وبيتحدثوا مع بعض ومتوافقين
- هنشئ اكثر من Container ونربطهم بنفس الـ Volume

```
yousef@DevOpsVM:~$ sudo docker run -d --name cont_vol1 -p 94:80 --volume appdata:/usr/local/apach  
e2/htdocs myapp:v1  
0d3b17a0f2ad1ac42457628522b308659eb36e4a7c880652cf2f509fe722667f  
yousef@DevOpsVM:~$ sudo docker run -d --name cont_vol3 -p 98:80 --volume appdata:/usr/local/apach  
e2/htdocs myapp:v1  
bf82ee7cf8304ebc9b0dac8c4cc14afbe65514e7902e76a507d1f5d755feb42b
```



- كدا أي تعديل على الـ Data من خلال الـ Volume

```
yousef@DevOpsVM:~$ sudo vim /var/lib/docker/volumes/appdata/_data/index.html
```

- هيسمع في كل الـ Containers في نفس الوقت